Feature Construction for Meta-heuristic Algorithm Recommendation of Capacitated Vehicle Routing Problems

HAO JIANG, YUHANG WANG, YE TIAN, and XINGYI ZHANG, Anhui University
JIANHUA XIAO, Nankai University

The algorithm recommendation is attracting increasing attention in solving real-world capacitated vehicle routing problems (CVRPs), due to the fact that existing meta-heuristic algorithms often show different performances on different CVRPs. To effectively perform algorithm recommendation for CVRPs, it becomes vital to extract suitable features to characterize the CVRPs accurately. To this end, in this article three groups of penetrating features are proposed to capture the characteristics of CVRPs. The first group consists of some basic features of CVRPs, where several features are suggested to capture the distribution of customer demand, the relationship between customer demand and vehicle capacity, besides some common attributes widely used in CVRPs. The second group is composed of the features extracted from some CVRP solutions generated by local search, where in addition to the feasible and better solutions, the worse solutions and the distribution of travel cost are also used to measure the sensitivity of CVRPs to local search operations. The third group is made up of image features obtained by depicting CVRP instances through images, which is first introduced by us to enhance the generalization of algorithm recommendation. Furthermore, based on the three groups of features, an algorithm recommendation method called ARM-I is built on the basis of a KNN classifier to recommend suitable algorithm for CVRPs. Experimental results on several selected benchmarks demonstrate the effectiveness of the designed features. More interestingly, the proposed ARM-I shows high generalization on real-world instances.

CCS Concepts: • **Applied computing → Transportation**; • **Computing methodologies → Feature selection**;

Additional Key Words and Phrases: Capacitated vehicle routing problem, algorithm recommendation, CVRP features, image features, meta-heuristic algorithms

ACM Reference format:
Hao Jiang, Yuhang Wang, Ye Tian, Xingyi Zhang, and Jianhua Xiao. 2021. Feature Construction for Meta-heuristic Algorithm Recommendation of Capacitated Vehicle Routing Problems. *ACM Trans. Evol. Learn. Optim.* 1, 1, Article 3 (April 2021), 28 pages.
https://doi.org/10.1145/3447540

This work was supported in part by the National Natural Science Foundation of China under Grant 61822301, Grant 61876123, Grant 62072258 and Grant U1804262, and the Anhui Provincial Natural Science Foundation under Grant 1808085J06.

Authors' addresses: H. Jiang, Y. Wang and X. Zhang (corresponding author), The Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei, China, 230601; emails: 19115@ahu.edu.cn, 1017507522@qq.com, xyzhanghust@gmail.com; Y. Tian, The Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, Institutes of Physical Science and Information Technology, Anhui University, Hefei, China, 230601; email: field910921@gmail.com; J. Xiao (corresponding author), The Research Center of Logistics, Nankai University, Tianjin, China, 300071; email: jhxiao@nankai.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2021 Association for Computing Machinery.
2688-299X/2021/04-ART3 $15.00
https://doi.org/10.1145/3447540

1 INTRODUCTION

As one of the most studied combinatorial optimization problems, vehicle routing problems (VRPs) are widely encountered in logistics, small package delivery, pickup services, and container terminal operations [Golden et al. 2008]. Capacitated vehicle routing problems (CVRPs) are a widely investigated variant of VRPs and has attracted attention of many researchers in recent decades.

The CVRPs aim to find the solutions to minimize the number of vehicles and the total travel cost while satisfying the customer demand and the constraint of vehicle capacity [Dantzig and Ramser 1959]. Various methods have been proposed for CVRPs, which can be divided into exact algorithms and inexact algorithms [Çağrı Koç et al. 2016]. Since CVRPs are NP-hard problems [Lenstra and Kan 1981], it is time-consuming to solve the instances with exact algorithms especially when the number of customers is large. Usually, the instances with size starting from 200 customers are considered as large-scale CVRPs [Li et al. 2005; Xiao et al. 2019]. To find quasi-optimal solutions in a reasonable time, a number of inexact algorithms have been devised to solve CVRPs [Oyola et al. 2017]. Although inexact algorithms cannot always obtain the optimal solution, they do not require large computational time. That is, the inexact algorithms sacrifice optimality to avoid large computational time. The inexact algorithms can be further divided into classical heuristic and meta-heuristic algorithms. The classical heuristic algorithms, such as savings algorithm [Clarke and Wright 1964], can quickly find a feasible CVRP solution, but there might be a large disparity between it and the best solution [Lin et al. 2009]. Different from classical heuristic algorithms, the meta-heuristic algorithms have ability of global search and usually provide satisfactory results in a reasonable time [Akpinar 2016]. Therefore, many meta-heuristic algorithms have been used to solve CVRP, such as ant colony optimization algorithms [Akpinar 2016; Kao et al. 2012; Sun et al. 2017], evolutionary optimization algorithms [Nazif and Lee 2012; Teoh et al. 2015], and particle swarm optimization algorithms [Chen et al. 2019; MirHassani and Abolghasemi 2011].

However, the quality of solutions obtained by meta-heuristic algorithms varies from one CVRP instance to another. Despite that a meta-heuristic algorithm shows promising performance on certain CVRP instances, its performance might considerately deteriorate on the other CVRP instances. According to the *No Free Lunch* theorem, there exists no such algorithm that outperforms other algorithms on all problem instances [Wolpert and Macready 1997]. Because of that, several researchers start to utilize algorithm recommendation to automatically select the most suitable algorithms from the algorithm library for different CVRP instances [Hutter et al. 2014; Kerschke et al. 2018; Mersmann et al. 2013]. The algorithm recommendation aims at predicting the performance of algorithm on a particular problem instance [Smith-Miles 2009]. It usually determines the relationship between the features of CVRP instances and the performance of algorithms, by which the algorithm recommendation method can select the best solving algorithm from the algorithm library.

In the above steps, the features used for characterizing CVRPs play an important role in algorithm recommendation [Kotthoff 2014]. A suitable feature set can help the algorithm recommendation method to distinguish different CVRP instances, which is the key to the performance of algorithm recommendation method. To this end, various features have been proposed to describe the characteristics of CVRPs [Kotthoff 2014]. On the whole, existing features used to characterize CVRP instances can be divided into two groups. The first group is the features extracted from the attributes of CVRP instances (called basic features), such as the location of customers, the vehicle capacity, and the customer demand. Rasku et al. [2019] designed certain features to describe the customer demand and vehicle capacity and combined them with some features tailored for TSPs to solve CVRPs. Kanda et al. [2016] used complex networks to depict CVRP instances and employed some network features to characterize CVRPs, such as the degree of the vertices and

edge reciprocity. Moreover, Mersmann et al. [2013] derived the features of CVRPs from distance matrix and spatial distribution of cities to measure the difficulty of CVRP instances for 2-opt. The second group is the features extracted from the solutions of CVRPs (called probing features). For example, Hutter et al. [2014] extracted some probing features from the solutions of a local search-based solver to reflect the sensitivity of CVRP instances [Lin and Kernighan 1973]. Smith-Miles and Hemert [2011] used feasible solution found by different greedy search operations to reflect the difficulty of CVRP instances.

Although various features have been extracted to characterize CVRPs, the task of characterizing CVRPs is still challenging. This is because in real-world application, the customer locations and demand are usually different from day to day, which requires more comprehensive features to depict CVRP instances, so when the CVRP instances change, the recommendation algorithm can still predict the best algorithm. However, the number of existing CVPR benchmark is limited, and they cannot reflect all the characteristics of real-world CVRP instances, which leads to a large difference between training data and real-world data. Although existing recommendation methods [Gutierrez-Rodríguez et al. 2019; Mayer et al. 2018; Rasku et al. 2019] achieve promising results on benchmark instances, their performance will considerably deteriorate on the real-world instances. Therefore, new features need to be designed so that the recommendation method can use the features extracted from benchmark instances to distinguish different real-world instances. In this article, we devise three groups of features to characterize CVRPs, including basic features, probing features, and image features.

For basic features, besides some widely used attributes of CVRPs, several features are proposed to describe the distribution of demand in detail and reflect the relationship between customer demand and vehicle capacity. This is because they can reflect the difficulty of CVRPs, which is helpful for distinguishing different CVRP instances. For probing features, since the worse solutions and the distribution of travel cost are seminal for measuring the sensitivity to local search, they are employed to characterize CVRPs in this article, in addition to widely used feasible and better solutions.

Apart from basic and probing features, certain image features are extracted to capture hidden characteristics of CVRPs via depicting CVRP instances by images. Recently, various methods have been designed to extract image features [Wong et al. 2017; Zhang et al. 2017; Zhao and Du 2016]. As global features, the image features can reflect color, texture, shape, and spatial relationship of items in the image. Due to the utilization of pixels in multiple areas, the image features can not only defense noise and local deviation but also are scale invariance, rotation invariance, and translation invariance. On account of these excellent properties, the image features are superior to basic and probing features, which are expected to be able to enhance the generalization of algorithm recommendation. Furthermore, based on the proposed features, we also devise an algorithm recommendation method called ARM-I by using a KNN classifier to select suitable algorithms for different CVRP instances.

To summarize, in this article, we design three groups of incisive features to comprehensively describe the characteristics of CVRPs, including basic features, probing features, and image features. For basic features, several new features are suggested to describe the distribution of customer demand, the relationship between customer demand and vehicle capacity, besides some common used basic features. For probing features, the worse solutions and the distribution of travel cost are used to depict the sensitivity of CVRP instances to local operations rather than only feasible and better solutions. The image features are first introduced by us to find certain characteristics widespread in CVRPs, which is able to improve the generalization of algorithm recommendation. According to the experimental results, all of these features can cooperatively characterize CVRPs and increase the generalization of algorithm recommendation methods.

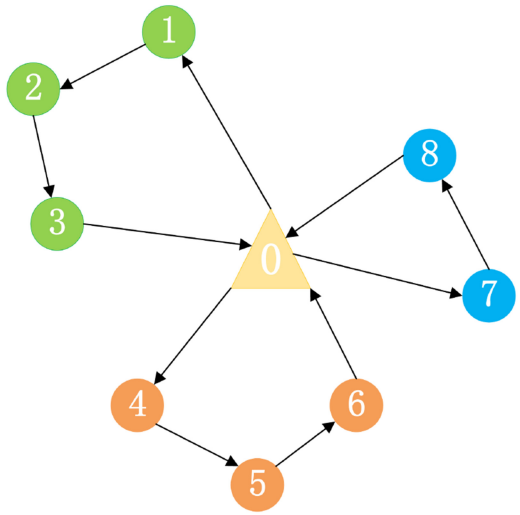


Fig. 1. An illustrative example of CVRP solution.

The remainder of this article is organized as follows. In Section 2, we introduce existing algorithms for solving CVRPs and existing features used to describe the characteristics of VRPs. Then, we elaborate on three groups of features tailored for CVRP instances in Section 3 and give the details of ARM-I in Section 4. The experimental results are shown in Section 5. In Section 6, we draw the conclusion of this article and discuss the future work.

2 RELATED WORK

In this section, the existing meta-heuristic algorithms for CVRPs are introduced. Moreover, the existing VRP features used for algorithm recommendation are also reviewed.

2.1 Existing Meta-heuristic Algorithms Tailored for Solving CVRPs

As a variant of VRP, CVRP has attracted a large number of researchers in recent decades. The purpose of CVRP is to find a delivery plan with the minimum travel cost, where all the vehicles start from and end at depot and all the customers are served and only once. Figure 1 gives a solution of CVRP, where the triangle point is depot and circle points represent the customers. The solution in Figure 1 contains three routes, which are "0-1-2-3-0," "0-4-5-6-0," and "0-7-8-0." Here, "0" represents depot, whereas the other numbers indicate customers. Nowadays, various meta-heuristic algorithms have been developed to generate high-quality solutions for CVRPs.

Seyyedhasani and Dvorak [2017] transformed agricultural field work into VRP and used improved Clark-Wright and tabu search to solve it, which significantly reduces the completion time. Besides, Ehmke et al. [2016] also used a tabu search algorithm to solve VRP, where the impact of traffic condition and the load on the vehicles are considered. Bouzid et al. [2017] presented an efficient Lagrangian relaxation method, which incorporates a new integer linear programming formulation and a variable neighborhood search to solve CVRP. Montoya et al. [2017] extended current electric vehicle routing problems models, where a nonlinear charging function is considered, and proposed a hybrid metaheuristic, where a local search operator is used to perturb the current solution and a global optimum is built by using parts of the local optima found during a heuristic search procedure. Eshtehadi et al. [2020] presented an enhanced adaptive

large neighborhood search algorithm, where new insertion and remove operators are developed to solve vehicle routing problem with multi-compartment vehicles.

Aside from neighborhood search, ant colony algorithm (ACO) is another popular technique for solving CVRPs [Akpinar 2016; Kao et al. 2012; Sun et al. 2017]. Kao et al. [Kao et al. 2012] combined ACO with particle swarm optimization and proposed a pheromone disturbance approach for CVRPs. Akpinar [2016] integrated ACO and large neighbourhood search to obtain satisfactory CVRP solutions. Sun et al. [2017] used SA as local search in ACO and devised a new pheromone perturbation strategy for CVRPs.

In addition, there are also several evolutionary optimization-based algorithms for CVRPs [Pierre and Zakaria 2017; Teoh et al. 2015; Ting et al. 2017]. Teoh et al. [2015] utilized differential evolution algorithm to solve CVRPs. Pierre and Zakaria [2017] designed a stochastic partially optimized cyclic shift crossover operator and used genetic algorithm to solve multi-objective vehicle routing problem with time windows. Ting et al. [2017] presented a multi-vehicle selective pickup and delivery problem and separately used three different algorithms to solve it, including tabu search, genetic algorithm, and scatter search, where four operators are devised to deal with route planning, the selection of pick up nodes, and the number of vehicles adopted.

Recently, several literature applied machine learning technology to solving CVRPs [Chen and Tian 2019; Kool et al. 2019; Nazari et al. 2018]. Nazari et al. [2018] presented an end-to-end framework using reinforcement learning to solve CVRP. Chen and Tian [2019] used reinforcement learning to automatically learn a local search operator for CVRP, and then the local search operator is employed to update the CVRP solution. Kool et al. [2019] used a pointer network with attention layer to solve CVRP. Moreover, certain new techniques have also been used for CVRPs in recent years, such as bat algorithm [Zhou et al. 2016] and symbiotic organisms search [Vincent et al. 2017].

Although various techniques have been used for CVRPs, there does not exist an algorithm outperforming other algorithms in all CVRP instances [Çağrı Koç et al. 2016]. For real-world CVRP instances, the customer location and demand are different every day. It is impracticable to run all solving algorithms to solve daily real-world CVRP instances. Therefore, the algorithm recommendation needs to be developed to select the most suitable algorithm for a given CVRP instance.

2.2 Existing Features for VRPs

The algorithm recommendation was first introduced to VRPs by Smith-Miles and Hemert [2011]. Since then, several effective algorithm recommendation methods were proposed for VRP [Hutter et al. 2014; Kerschke et al. 2018; Mersmann et al. 2013]. To recommend algorithm for VRP instances, numerous features need to be extracted from problem instances. Therefore, various features have been designed for characterizing VRP instances [Kotthoff 2014; Mayer et al. 2018].

The first group of features for VRP instances are extracted from the attributes of VRP instances, which can be roughly divided into following three categories. The first category is the customers distribution-based features. They are usually derived from the distance between different customers and their demand, such as the mean of demand [Rasku et al. 2019] and variance of distance matrix [Zhang and E. Korf 1996], the central node of normalized distance matrix [Hutter et al. 2014], the convex hull area composed of all customers (the proportion of customers on the convex hull, the distance between customers in the area and the edge of area) [Markus et al. 2018; Olaf et al. 2013], and the rectangular area covering all customers [Smith-Miles and Hemert 2011]. The second category is cluster-based features, which are extracted from the clusters of customers, such as the number of clusters and the distance between different clusters [Smith-Miles et al. 2010]. The third category is graph-based features. These features are usually drawn from spanning trees or networks of customers. For example, Olaf et al. [2013] built a minimum spanning

tree of customers and used the minimum, mean, median, maximum, and the standard deviation of the depth and distance values of the minimum spanning tree to characterize VRP instances. Pihera and Musliu [2014] constructed a k-nearest-neighbor graph for customers and employed strongly connected components and weakly connected components of networks to describe the characteristics of VRPs. Kanda et al. [2016] extracted some features from the networks of customers, such as maximum degree of vertices, average geodesic distance, and network cyclic coefficient, to characterize VRP instances.

The second group of features are extracted from the solutions of VRP instances [Kerschke et al. 2018], which are used to measure the sensitivity of CVRPs to local search operations. The solutions for feature extraction are usually generated by different local search operations. For example, Gutierrez-Rodríguez et al. [2019] combined some perturbative heuristic algorithms, such as 2-opt, exchange, and relocate, with randomized hill-climbing search to generate solutions, and took the mean and the standard deviation of found feasible solutions as features. Hutter et al. [2014] employed a local search-based solver proposed in Lin and Kernighan [1973] to generate solutions and used tour length and tour quality improvement to describe the characteristic of VRP instances. Mersmann et al. [2012] used the hardness of instances for 2-opt operator to characterize VRP instances.

On the whole, although various features have been used for recommending algorithms for VRPs, it is still challenging to characterize VRP instance, which results in the low generalization of existing algorithm recommendation methods. Although they achieve promising performance on several selected benchmarks, they encounter difficulty in the real-world instances. Additionally, none of them uses image features to characterize VRP instances.

3 THREE GROUPS OF FEATURES

An appropriate feature set is crucial for recommendation algorithm. Different features will significantly affect their performance. In this article, we use three groups of features to comprehensively capture the characteristics of CVRPs, including basic features, probing features, and image features.

3.1 Basic Features

In this article, the basic features refer to the features obtained from certain attributes of CVRP instances, such as position of customers and depot, the vehicle capacity, and the customer demand. The basic features used in this article are listed in Table 1, where the basic features designed by us are highlighted. In Table 1, basic features 1–9 are directly derived from basic attributes of the CVRP instances, whereas basic features 10–16 are derived from the customer clusters in CVRP instances. In Table 1, CV and VR are two metrics used to measure the extent of customer cluster violating the capacity constraint, which can be calculated by Equations (2) and (3). Here, we use k-means clustering to obtain the customer clusters. The k is calculated as follows:

$$k = \left\lceil \frac{\sum_{i=1}^{n} d_i}{c} \right\rceil, \tag{1}$$

where d_i is demand of customer i and c is the vehicle capacity. Essentially, k is the minimum number of vehicles required for serving all the customers. For two solutions with same travel cost, the one using less vehicles is better than the other. Therefore, we use the minimum vehicle number to determine the number of customer clusters.

For features 1–6, they are often used to capture a coarse characteristic of CVRP instances [Arnold and Sorensen 2019; Gutierrez-Rodríguez et al. 2019; Hutter et al. 2014]. However, they belittle the role of distribution of customer demand and only use the mean of customer demand. The mean

Table 1. Definition of Basic Features

Number	Feature	Description
1	$mean(Dis)$ [Hutter et al. 2014]	Mean of the distance among customers.
2	$std(Dis)$ [Hutter et al. 2014]	Standard deviation of the distance among customers.
3	$max(Dis)$ [Hutter et al. 2014]	Max of the distance among customers.
4	$median(Dis)$ [Hutter et al. 2014]	Median of the distance among customers.
5	$Capacity$ [Gutierrez-Rodríguez et al. 2019]	The vehicle capacity.
6	$mean(Demand)$ [Gutierrez-Rodríguez et al. 2019]	Mean of the customer demand.
7	$\boldsymbol{max(Demand)}$	Maximum of the customer demand.
8	$\boldsymbol{median(Demand)}$	Median of the customer demand.
9	$\boldsymbol{skew(Demand)}$	Skewness of the customer demand.
10	$mean(Cen2depot)$ [Hutter et al. 2014]	Mean of the distance from cluster center to depot.
11	$\ln(std(Cen2depot))$ [Hutter et al. 2014]	Natural logarithm of standard deviation of the distance from cluster center to depot.
12	$std(mean(Cus2cen))$ [Hutter et al. 2014]	Standard deviation of mean of the distance from customers to cluster center.
13	$\boldsymbol{sum(CV)}$	Sum of the value to which these clusters violates vehicle capacity constraints.
14	$\boldsymbol{\ln(mean(CV))}$	Natural logarithm of mean of the value to which these clusters violates vehicle capacity constraints.
15	$\boldsymbol{\ln(std(CV))}$	Natural logarithm of standard deviation of the value to which these clusters violates vehicle capacity constraints.
16	$\boldsymbol{sum(VR)}$	Sum of the extent to which these clusters violates vehicle capacity constraints.

of customer demand is impotent to characterize the distribution of demand. This is because two CVRP instances with different distribution of customer demand are likely to have the same mean of customer demand. As an important basic attribute of CVRP instances, the distribution of customer demand needs to be emphasized. Therefore, we additionally leverage the maximum, median, and skewness of customer demand, namely basic features 7–9, to thoroughly capture the distribution of demand.

For basic features 10–12, they characterize CVRP instances from the view of customer clusters, which are widely used in Hutter et al. [2014], Mersmann et al. [2013], and van Hemert [2005]. The relationship between customer demand and vehicle capacity can reflect the difficulty of solving CVRP instances, which is beneficial to preciously distinguishing different CVRP instances. Hence, we define two metrics CV_x and VR_x to measure the extent of customer cluster x violating the capacity constraint,

$$CV_x = max\{D_x - c, 0\}, \tag{2}$$

$$VR_x = \frac{D_x}{c}, \tag{3}$$

where D_x is the sum of customer demand in cluster x and c is the vehicle capacity. Here, CV_x and VR_x are used to measure the difficulty of CVRP instances. For an instance, if each CV_x is close to zero and each VR_x is close to 1, then this instance can be regarded as a simple one. Because in this case we can divide customers into different groups by simple clustering and then arrange the

Table 2. Definition of Probing Features

Number	Feature	Description
1	N_f [Gutierrez-Rodríguez et al. 2019]	The number of feasible solutions.
2	N_b [Hutter et al. 2014]	The number of feasible solutions that are better than initial solution.
3	$\boldsymbol{mean(cost)}$	Mean of travel costs.
4	$\boldsymbol{std(cost)}$	Standard deviation of travel costs.
5–7	$\boldsymbol{Q}$	25%, 50%, 75% quartile of travel costs.
8	$\boldsymbol{OR}$	Outlier ratio, the ratio of worst travel cost and best travel cost.

service order for customers in each group. Due to numerous customer clusters in a CVRP instance, we devise basic features 13–16 based on CV_x and VR_x to describe the difficulty of CVRP instance.

3.2 Probing Features

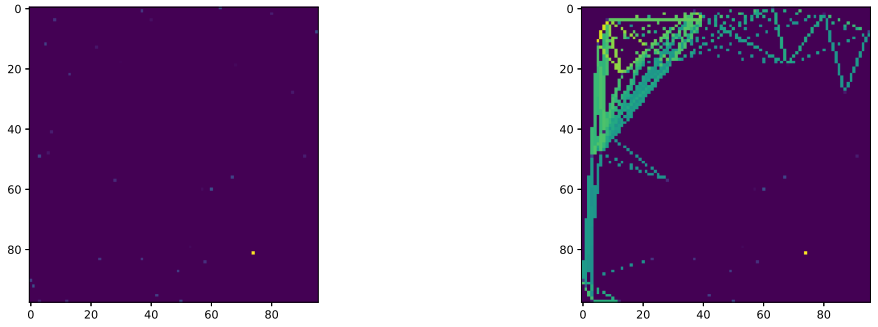
We refer to the features obtained from the solutions of CVRP instances as probing features. A solution generated by local search contains ample information about CVRP instance, which can reflect the sensitivity of CVRP instance to local search operations. The sensitivity of CVRP instance to local search operations can guide the procedure of recommendation. Therefore, probing features should also be used to describe the characteristics of CVRPs. In this article, we use three common local search operations including $2-opt$, $swap$, and $insert$ to generate 1,000 solutions for each CVRP instance. Specifically, for an instance, we first randomly generate a feasible solution, and then randomly select a local search operation to generate a new solution. The probing features used in this article are list in Table 2, where the probing features proposed in this article are highlighted.

The number of feasible solutions, and among them, the number of solutions, which are better than initial solution, can roughly reflect the sensitivity of a CVRP instance to local search operations. Therefore, probing features 1 and 2 are employed to describe the characteristics of CVRPs. However, features 1 and 2 are narrow. It is obvious that two instances with the same N_f and N_b but different change on travel cost have different sensitivity to local search operations. Here, the increase and decrease of travel cost are taken into account. The more change on travel cost of generated solutions is, the more sensitive a CVRP instance is to local search operations. Therefore, the distribution of travel cost is noteworthy, and we develop features 3–8 to reveal the distribution of travel cost of generated solutions in detail.

3.3 Image Features

We refer to the features extracted from images as image features. The motivation of using image features is that it can capture some superior characteristics. Since image features involve pixels of multiple areas in the image, they have rotation invariance and strong resistance to noise and local deviation. Due to these excellent properties, the utilization of image features can detect some hidden characteristics of CVRPs, and enhance generalization of algorithm recommendation. Therefore, in this article, we use an image to depict a CVRP instance, and extract certain image features to characterize the CVRP instance.

To use an image to depict a CVRP instance, we first use a rectangular image to cover all the customer nodes and depot. The length l and width w of the rectangular image are defined as



(a) Before adding edges. (b) After adding edges.

Fig. 2. The example of CVRP image. The blue dots represent customers, and the yellow dots is the depot.

follows:

$$\begin{cases} l = \lfloor x_{max} - x_{min} + 1 \rfloor \\ w = \lfloor y_{max} - y_{min} + 1 \rfloor \end{cases}, \tag{4}$$

where x_{max} and x_{min} are the maximum and minimum of x-coordinate of depot and customer locations and y_{max} and y_{min} are the maximum and minimum of y-coordinate of customer locations. Note that for some real-world instances, their coordinate are latitudes and longitudes. In this case, before converting CVRP instances into images, we need to convert their x-coordinates and y-coordinate into [0,100] according to the latitudes and longitudes.

Then, we map each customer location to a pixel in the rectangular image, where the coordinate of each customer (P_{ix}, P_{iy}) can be calculated by Equation (5),

$$\begin{cases} P_{ix} = \lfloor x_i - x_{min} \rfloor \\ P_{iy} = \lfloor y_i - y_{min} \rfloor \end{cases}, \tag{5}$$

where x_i is the x-coordinate of customer i and y_i is the y-coordinate of customer i.

Next, we use the pixel values to reflect the customer demand, where the the pixel value of point (i, j) can be determined by Equation (6),

$$p_{ij} = \begin{cases} \frac{d_x}{c}, & if\ (i,j)\ represents\ customer\ x \\ 1, & if\ (i,j)\ represents\ depot \\ 0, & otherwise \end{cases}. \tag{6}$$

Here, d_x is the demand of customer x and c is the vehicle capacity.

After above steps, we can obtain an image to depict a CVRP instance such as Figure 2(a). However, as can be seen from Figure 2(a), the most pixels have a value of 0 (black dots), and the number of pixel points associated with customers (blue dots) is much fewer. That is, most pixels in the image are meaningless. To increase the number of pixels associated with customers, we use saving distance [Clarke and Wright 1964] between customers to add some edges to image.

First, we use Equation (7) to calculate the saving distance between each pair of customers,

$$save_{ij} = d_{0i} + d_{j0} - d_{ij}, \tag{7}$$

where d_{0i} and d_{j0} are the distance between customers i and j and depot and d_{ij} is the distance between customers i and j. Second, we normalize the saving distance between each pair of customers. Third, we add edges between the pairs of customers whose normalized saving distance is greater than 0.5. Here, the values of pixels associated with the edges are set to the normalized saving distance between customers. After adding edges, we can obtain an image like Figure 2(b)

to depict a CVRP instance, where more pixels are associated with customers (green dots). In above process, the information of customer locations, customer demand, and vehicle capacity are all used to generate the image. Therefore, the generated image can delineate a CVRP instance.

After obtaining the images, we use the LOSIB [García-Olalla et al. 2014], which is an image feature extraction method, to extract image features. The code of LOSIB is available at https://github.com/annikaliebgott/ImFEATbox. The LOSIB considers the pixel value differences along several orientations as image features. By using LOSIB, we totally obtain 17 image features. To be specific, the first 16 features extracted by LOSIB are the combination between textural information and statistical information of pixels in different orientation. In other words, the LOSIB uses the mean value of pixel differences between each pixel and its neighbors, where the neighbors of a pixel are the pixels in a certain orientation within a certain radius. In this article, we analyze the pixel differences in 16 orientation, and obtain the first 16 image features. Moreover, the LOSIB also calculates the standard deviation of the first 16 features, and use it as the 17th feature. Because when converting a CVRP instance into an image, we map each customer and depot to a pixel in the rectangular image, and use customer demand as the pixel values, the image features extracted by LOSIB can capture both of the relative locations of different customers and the difference of their demand.

4 THE PROPOSED ARM-I

Based on the proposed three groups of features, we design a method called ARM-I to recommend suitable solving algorithms for given CVRP instances. The procedure of ARM-I is shown in Algorithm 1. The Algorithm 1 has three parameters, which are the set of CVRP instances for training ($Train$), the set of CVRP instances to be predicted (Pre), and the algorithm library (AL), separately. The ARM-I first extracts three groups of features FTr_{basic}, $FTr_{probing}$, and FTr_{image} from the training CVRP instances and then standardizes these features by z-score (lines 1–3). More concretely, for a feature x, its normalized features x_{new} can be calculated by the following equation:

$$x_{new} = \frac{x - \mu}{\sigma}, \tag{8}$$

where μ and σ are the mean and the standard deviation of all features, respectively.

When extracting probing features, the solutions are randomly generated. For the purpose of reducing the randomness of probing features, we conduct the procedure of extracting probing features 10 times and use the average result as the final probing features. Moreover, since the image features used in ARM-I are the average pixel difference between each pixel and its neighbors in 16 different orientations, some CVRP instances with different best solving algorithms may have similar image features. Therefore, too much reliance on image features is likely to result in a wrong recommendation. To this end, we set the weights to basic and probing features (line 4) to help the ARM-I distinguishing different types of instances and increase its performance. Specifically, we use the extracted basic and probing features multiply w as final basic and probing features, separately,

$$w = round\left(\frac{n_{all}}{n_{basic} + n_{probing}}\right), \tag{9}$$

where $round(a)$ is the result of rounding a, n_{all} is the number of features, n_{basic} is the number of basic features, and $n_{probing}$ is the number of probing features.

After that, the ARM-I assigns the algorithm with the best performance to each CVRP instance (lines 5–10). Because in algorithm library AL, algorithms are meta-heuristic, which might obtain different results for different runs, we run all algorithms num times for each CVRP instance and then use the average result m to evaluate the performance of each algorithm [Chu et al. 2019].

ALGORITHM 1: The procedure of predicting best algorithms

Input: $Train$: the set of CVRP instances for training; Pre: the set of CVRP instances to be predicted; AL: the algorithm library.
Output: A: the predicted best algorithm for each instance in Pre

1 $FTr_{basic} \leftarrow$ Calculate basic features from $Train$ and perform standardization;
2 $FTr_{probing} \leftarrow$ Calculate probing features from $Train$ and perform standardization;
3 $FTr_{image} \leftarrow$ Calculate image features from $Train$ and perform standardization;
4 $F_{train} \leftarrow (FPr_{basic} \times w) \cup (FPr_{probing} \times w) \cup FPr_{image}$;
5 $label \leftarrow \emptyset$;
6 **for** $i = 1$ $to|Train|$ **do**
7 $\quad m \leftarrow \emptyset$;
8 $\quad$ **for** $j = 1$ $to|AL|$ **do**
9 $\quad\quad m_j \leftarrow$ Calculate the average result of AL_j;
10 $\quad label_i \leftarrow$ Select the index of algorithm with mininum metric m;
11 $FPr_{basic} \leftarrow$ Calculate basic features from Pre and perform standardization;
12 $FPr_{probing} \leftarrow$ Calculate probing features from Pre and perform standardization;
13 $FPr_{image} \leftarrow$ Calculate image features from Pre and perform standardization;
14 $F_{Pre} \leftarrow (FPr_{basic} \times w) \cup (FPr_{probing} \times w) \cup FPr_{image}$;
15 $A \leftarrow KNN(F_{train}, label, F_{Pre})$;
16 **return** A;

Given that there exists more than one solution in the search process, the smaller the value of m the better performance the algorithm achieves on the instance. As our goal is to select the algorithm with the best performance for each CVRP instance, after obtaining the metric m of all algorithms, we just need to select the algorithm with the smallest m as the label of the CVRP instance.

At last, a KNN classifier is employed to predict the best algorithm for a given CVRP instance (lines 11–15). Here, $k = 3$. Because for a CVRP instance, the algorithm in library with the best performance can be viewed as the label of the instance, predicting the best algorithm for a given CVRP instance can be viewed as a classification problem [Chu et al. 2019; Gutierrez-Rodríguez et al. 2019]. In this article, we use a KNN classifier to divide the instances in Pre into different classes to predict the best algorithms.

5 EMPIRICAL STUDIES

In this section, the developed features and ARM-I are tested on several selected benchmark and real-world instances. We first introduce the algorithm library, evaluation metrics and datasets used in the experiments. Then, we show the performance of ARM-I from two aspects including accuracy and generalization. Next, the effectiveness of the features, the weight and the image feature extraction method used by ARM-I are verified. Finally, the parameters sensitivity of ARM-I is analyzed.

5.1 Algorithm Library, Evaluation Metrics, and Dataset

5.1.1 Algorithm Library. In simulation experiments, the algorithm library consists of six algorithms including GTS [Toth and Vigo 2003], SATS [Lin et al. 2009], IVND [Chen et al. 2010], PACO [Kao et al. 2012], EABC [Szeto et al. 2011], and SALS [Alabas-Uslu and Dengiz 2011]. Among these six algorithms, GTS, SATS, IVND, and SALS are local search-based algorithms. GTS is a tabu search-based method, where a new neighbor structure is defined and the tabu search is used to find better solutions. SATS uses the cooling mechanism of SA to control the iteration in tabu search

and embeds local operators *swap* and *insert* into algorithm. SALS uses five different local search operators and considers a small subset of the neighborhood instead of the whole search neighborhood. IVND combines iterated local search with variable neighborhood descent and designs a perturbation strategy to escape from local minima by cross-exchange operator. PACO and EABC are population-based algorithms. PACO is a hybrid algorithm of ant colony algorithm and particle swam algorithm. In PACO, the ant can be viewed as a particle that can memorize the best solution that was found (in the past) and only elite ants are allowed to update pheromone according to their best-so-far solution. EABC improves the artificial bee colony algorithm by changing the updated strategy of neighbor solution.

The parameters used in these six algorithms are configured according to the recommended values in the original papers. All the algorithms in the library are re-implemented by us in Python. Because different solving algorithms usually require different running time to find better solutions. In this article, we are more focused on the performance of the recommendation algorithm. To ensure that all solving algorithms have sufficient search time, in the experiments, the stopping criteria of each algorithm is that the best solution found by it has not been updated for 50 generations.

5.1.2 Evaluation Metrics. In this article, the precision of predicted results (*Precision*), average rank (*AvgRank*), and mean reciprocal rank (MRR) are employed as the metrics to evaluate the performance of recommendation method. The *Precision* is the proportion of the best algorithm in the predicted results, which can be calculated as follows:

$$Precision = \frac{n_{best}}{N} \times 100\%, \tag{10}$$

where n_{best} is the number of the instances for which the predicted algorithms are the best algorithms, whereas N is the number of instances. The objective of algorithm recommendation method is to select the best algorithm for each instance. Therefore, the closer the *Precision* to 100% the better the method is.

However, selecting the best algorithm for each instance is a tough task. Sometimes, it is acceptable to select algorithm with better performance. Therefore, it is inadequate to use Precision alone. Thus, we also use the *AvgRank*, which is the average rank of predicted results on all CVRP instances, as the performance metric. Here, the candidate list of the average rank is the algorithm in the algorithm library. The *AvgRank* can be calculated as follows:

$$AvgRank = \frac{\sum_{i=1}^{N} rank_i}{N}, \tag{11}$$

where $rank_i$ is the predicted result's rank of instance i. The closer the *rank* to 1 the better the method is.

The MRR is a metric widely used to evaluate recommendation algorithm [Craswell 2009; Wu and Yang 2008], which is the mean reciprocal of the rank at which the first relevant document is retrieved. Since the ARM-I only outputs one solving algorithm for a given CVRP instance, we use the mean reciprocal of the rank at each instance to calculated MRR. That is, it can be calculated by Equation (12). The closer the MRR to 1 the better the method is,

$$MRR = \frac{1}{N} \sum_{i=1}^{N} \frac{1}{rank_i}. \tag{12}$$

5.1.3 Datasets. To verify the performance of the ARM-I, we use certain common instance sets of CVRPs including *A* [Augerat 1995], *B* [Augerat 1995], *CMT* [Christofides 1976], *E* [Christofides and Eilon 1969], *F* [Fisher 1994], *M* [Christofides 1976], *P* [Augerat 1995], and *X* [Uchoa et al. 2017].

Table 3. Performance of the Algorithms in Algorithm Library on Benchmark Instances

Algorithm	*percentage*	*average rank*
EABC	5.3% (32/600)	5.08
SALS	7.4% (44/600)	3.48
GTS	5.3% (32/600)	4.53
IVND	22% (132/600)	2.69
SATS	**44%** (264/600)	**2.04**
PACO	16% (96/600)	3.16

Here, we only select the instances of which number of customers is less than 300, given that the algorithms in algorithm library are not designed for large-scale CVRPs. Then, we obtain 150 CVRP instances. For increasing the number of CVRP instances, we rotate the coordinate of customers and depot around coordinate (0, 0) by 90°, 180°, and 270° separately and generate three extra instances for each CVRP instance. Finally, we obtain 600 instances. The performance of the algorithms on 600 CVRP instances is shown in Table 3, where "percentage" means the proportion of the algorithm performing best on 600 instances, whereas "average rank" means the average rank of the solutions outputted by the algorithm. Here, each algorithm is run 10 times on each CVRP instance, namely $num = 10$. From Table 3, we can obtain that the SATS is the baseline in simulation experiments. The baseline indicates the algorithm with the best performance in algorithm library. An algorithm recommendation method is meaningful only if its performance is better than the baseline.

Furthermore, we also use three real-world instance sets. The first set contains 3 instances, where the cities in Belgium are used as customers with appropriate customer demand and vehicle capacity.[1] The second set contains 9 instances, where the data are provided by a company from Spain [Castro-Gutierrez et al. 2011]. In the second set, the company delivers food products and serves customers during a period of time, which can be seen as the VRPTW. Here, we ignore the time windows of customers and finally obtain 9 instances. The third set contains 16 instances, which is provided by a company in the Canary Islands [Armas et al. 2015].[2] The size of these real-world instances are given in Table 4.

The performance of algorithms in algorithm library on these real-world instances is shown in Tables 5 and 6. The Table 5 displays the rank of algorithms in algorithm library on real-world instances, where we rank them by the average results of 10 runs. The Table 6 shows two statistical indicators of the performance of algorithms in algorithm library on real-world instances including *percentage* and *average rank*. The *percentage* means the proportion of solving algorithm that ranks first on all instances, whereas *average rank* is the average value of the rank of solving algorithm on all instances. Here, we can find that for above 28 real-world CVRP instances, the algorithm IVND is the baseline.

5.2 Performance of ARM-I

5.2.1 Accuracy. The purpose of recommendation method is to predict the best solving algorithm for a given instance. Therefore, we first show the accuracy of ARM-I on benchmark instances. To show the superiority of the proposed ARM-I, we also run the other three algorithm recommendation methods including SBAAS [Mayer et al. 2018], ASTW [Gutierrez-Rodríguez et al. 2019], and ASC [Rasku et al. 2019] on benchmark instances. All of these three methods are

[1] https://www.optaplanner.org/blog/2014/09/02/VehicleRouting/WithRealRoadDistances.html.
[2] https://sites.google.com/site/gciports/vrptw/hfvrptw.

Table 4. The Size of Real-world Instances

Real-world instance	Number of customers	Real-world instance	Number of customers
belgium-n50-k10	50	n1_3	100
belgium-n100-k10	100	n1_4	100
belgium-n500-k20	500	n2_1	100
t-50-d0	50	n2_2	100
t-50-d1	50	n2_3	100
t-50-d2	50	n2_4	100
t-150-d0	150	n3_1	100
t-150-d1	150	n3_2	100
t-150-d2	150	n3_3	100
t-250-d0	250	n3_4	100
t-250-d1	250	n4_1	100
t-250-d2	250	n4_2	100
n1_1	100	n4_3	100
n1_2	100	n4_4	100

re-implemented by us in Python. It is worth noting that SBAAS and ASTW are designed for more complex variant of VRPs, and in this article we only retain the features that are relevant to CVRPs. In the simulation experiment, we use 10-fold crossing validation strategy to verify the performance of different algorithm recommendation methods. To ensure the reliability of experimental results, the experiment is independently run 10 times. The average experimental results are shown in Table 7.

As can be seen from Table 7, the performance of ARM-I and ASC are much better than the baseline algorithm, whereas the performance of SBAAS and ASTW are close to the baseline algorithm. Although the ARM-I performs slightly worse than *ASC*, it still obtains a satisfactory *Precision*, *AvgRank*, and *MRR* that are much greater than the baseline. The relatively poor performance of *SBAAS* and *ASTW* is partly due to the fact that they only extract 8 and 18 features to depict the characteristics of CVRP instances, separately. The number of features is too small to distinguish different instances. Therefore, both of *SBAAS* and *ASTW* do not perform very well on benchmark instances. Different from *SBAAS* and *ASTW*, the *ASC* extracts 433 features to describe instances. Therefore, the *ASC* achieves a satisfactory performance, much better than baseline. As for our ARM-I, it extracts 41 features to depict the characteristics of CVRP instances, and the features extracted by it can capture certain characteristics widespread in CVRP instances. Therefore, although it use 41 features far less than *ASC*, the ARM-I still achieves a competitive performance on benchmark instances.

Moreover, we also use Wilcoxon rank sum test (with significant level 0.05) to verify whether there is significant difference between the rank of solving algorithm predicted by ARM-I and the other recommendation methods. The experimental results are shown in Table 8, where "+" means that there is a significant difference between the rank of solving algorithm, whereas "∼" indicates that there is no significant difference. From Table 8, we can find that the performance of ARM-I has significant difference from *SBAAS* and *ASTW*, whereas there is no significant difference between ARM-I and *ASC*. Although *ASC* achieves better *Precision*, *AvgRank*, and *MRR* than ARM-I, the rank of solving algorithm predicted by them has no significant difference. Therefore, ARM-I owns a competitive performance on benchmark instances.

Table 5. Rank of Algorithms in Algorithm Library on Real-world Instances

Real-world instance	*EABC*	*SALS*	*GTS*	*IVND*	*SATS*	*PACO*
belgium-n50-k10	4	2	5	3	1	6
belgium-n100-k10	4	2	6	3	1	5
belgium-n500-k20	4	6	3	2	1	5
t-50-d0	1	2	6	3	5	4
t-50-d1	3	2	6	1	5	4
t-50-d2	6	1	4	2	5	3
t-150-d0	5	1	4	2	6	3
t-150-d1	6	1	4	2	5	3
t-150-d2	2	3	5	1	4	6
t-250-d0	5	2	3	1	6	4
t-250-d1	6	2	4	1	5	3
t-250-d2	5	6	2	1	4	3
n1_1	2	6	4	1	3	5
n1_2	2	6	3	1	4	5
n1_3	2	6	5	1	3	4
n1_4	3	6	2	1	4	5
n2_1	2	6	3	1	4	5
n2_2	2	6	3	1	4	5
n2_3	2	6	3	1	4	5
n2_4	3	6	2	1	4	5
n3_1	2	6	3	1	5	4
n3_2	2	6	4	1	3	5
n3_3	2	6	3	1	4	5
n3_4	3	6	2	1	4	5
n4_1	3	6	2	1	5	4
n4_2	2	6	5	1	3	4
n4_3	2	6	3	1	4	5
n4_4	3	6	2	1	4	5

Table 6. Performance of Algorithms in Algorithm Library on Real-world Instances

Metric	*EABC*	*SALS*	*GTS*	*IVND*	*SATS*	*PACO*
percentage	3.57%(1/28)	10.71%(3/28)	0%(0/28)	**75%**(21/28)	10.71%(3/28)	0%(0/28)
average rank	3.14	4.50	3.61	**1.36**	3.93	4.46

Table 7. *Precision*, *AvgRank*, and *MRR* of Algorithm Recommendation Methods on Benchmark Instances

Recommendation Methods	*Precision*	*AvgRank*	*MRR*
SBAAS	49.5%(297/600)	1.9	0.69
ASTW	36%(252/600)	2.28	0.6
ASC	**92.8%**(557/600)	**1.11**	**0.96**
ARM-I	89.1%(535/600)	1.22	0.93

Table 8. The Wilcoxon Rank Sum Test Results between ARM-I and the Other Recommendation Methods on Benchmark Instances

Benchmark instances	*SBAAS*	*ASTW*	*ASC*
ARM-I	+	+	~

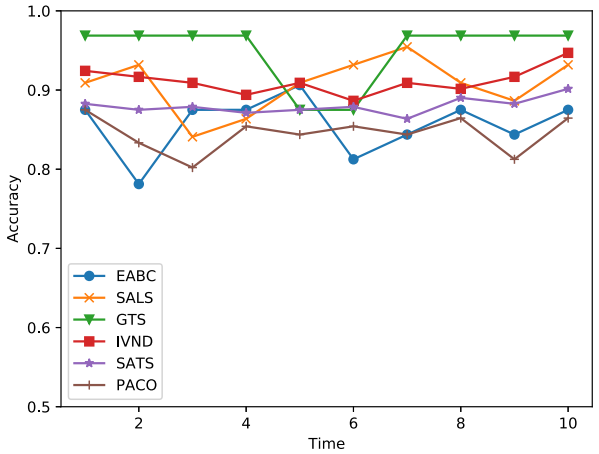


Fig. 3. The predicted precision of ARM-I for each algorithm in algorithm library.

To comprehensively show the predicted results of ARM-I, we divide CVRP instances into six groups according to their best algorithms. Then, we calculate the *Precision* of ARM-I for each group and plot them in Figure 3. Figure 3 depicts the precision of 10 times results predicted by ARM-I. To plot Figure 3, we first divide all instances into six groups according to their best algorithms. Then, the ARM-I is employed to predict the best algorithm for each instance. Finally, we calculate the prediction precision for each group and show them in Figure 3. From Figure 3, we can see that the *Precision* of ARM-I in all all group is stably greater than 80%, which means that our method can distinguish the performance of all algorithms rather than only predict some algorithms that perform well on training set.

5.2.2 Generalization. For a recommendation method, its generalization is also an important aspect to evaluate the performance, which measures the adaptability to fresh instances. To validate the generalization of ARM-I, we use the model of ARM-I built on benchmark instances to predict the best algorithm for real-world CVRP instances. Table 9 gives the *Precision*, *AvgRank*, and *MRR* of each algorithm recommendation method on real-world instances. From Table 9, we can obtain the following two observations.

First, the proposed ARM-I is still effective in real-world problems. In Table 9, the *Precision* of ARM-I is 85.7% much greater than the percentage of IVND (75%), and the *AvgRank* is 1.21 also better than the rank of IVND (1.36). That is, the performance of ARM-I is much better than the baseline. Therefore, the ARM-I is effective in real-world problems.

Second, the ARM-I owns better generalization than the other three algorithm recommendation methods. On the benchmark instances, all of the three compared algorithm recommendation methods obtains pretty good results. However, the performance of them considerately deteriorates

Table 9. *Precision*, *AvgRank*, and *MRR* of Algorithm Recommendation Methods on Real-world Instances

Metric	*SBAAS*	*ASTW*	*ASC*	*ARM-I*
Precision	10.7%(3/28)	10.7%(3/28)	7.1%(2/28)	**85.7%**(24/28)
AvgRank	3.93	3.93	3.96	**1.21**
MRR	0.32	0.32	0.31	**0.92**

Table 10. The Wilcoxon Rank Sum Test Results between ARM-I and the Other Recommendation Methods on Real-world Instances

Real-world instances	*SBAAS*	*ASTW*	*ASC*
ARM-I	+	+	+

Table 11. Effectiveness of Three Groups of Features on Benchmark Instances

Metrics	Basic features	Probing features	Image features
Precision	68.0%	84.0%	86.2%
AvgRank	1.63	1.31	1.28
MRR	0.8	0.9	0.91

on the real-world instances. All of them are worse than the baseline on the real-world instances. Nevertheless, the ARM-I always performs satisfactorily far better than baseline.

The reason why the *SBAAS* and *ASTW* achieve relatively poor performance on real-world instances is still that they use few features to characterize CVRP instances. In Table 9, the performance of *ASC* considerably deteriorates on real-world instances. As mentioned above, *ASC* extracts 433 features. Although the *ASC* employs a method of dimension reduction to reduce the number of features, it still uses 75 features to describe instances. Obviously, the *ASC* uses too many features, which have negative impact on generalization of recommendation method. Therefore, although it achieves promising performance on benchmark instances, its performance considerably deteriorates on real-world instances. Different from *SBAAS*, *ASTW*, and *ASC*, our ARM-I uses 41 features to describe the characteristics of CVRP instances, which achieves a better balance between accuracy and generalization.

Besides, Table 10 gives the Wilcoxon rank sum test result (with significant level 0.05) between ARM-I and three recommendation methods on real-world instance, where the performance of ARM-I has significant difference from the other three recommendation methods. Therefore, the ARM-I has a better generalization than the other three recommendation methods.

5.3 Effectiveness of the Features

5.3.1 Effectiveness of Features Used by ARM-I on Accuracy. The ARM-I uses three groups of features (41 features). To verify the effectiveness of features used by ARM-I, we discard each group of features separately and record the accuracy of ARM-I on benchmark instances. The experimental results are presented in Table 11. As shown in Table 11, the metrics of ARM-I will get worse no matter which group of features we discard. Particularly, without basic features, the *Precision* of ARM-I deteriorates from 89.1% to 68.0%, its *AvgRank* deteriorates from 1.22 to 1.63, and its *MRR*

Table 12. Effectiveness of Basic Features on Benchmark Instances

Basic Features	*Precision*	*AvgRank*	*MRR*
1	87.5%	1.24	0.92
2	88.3%	1.26	0.92
3	88.3%	1.23	0.93
4	89.0%	1.23	0.93
5	88.7%	1.24	0.93
6	89.3%	1.22	0.93
7	87.5%	1.26	0.92
8	87.7%	1.25	0.92
9	88.2%	1.25	0.93
10	86.8%	1.27	0.92
11	86.5%	1.28	0.91
12	88.2%	1.24	0.93
13	87.5%	1.28	0.92
14	88.7%	1.23	0.93
15	87.3%	1.26	0.92
16	86.5%	1.28	0.92

Table 13. Effectiveness of Probing Features on Benchmark Instances

Probing Features	*Precision*	*AvgRank*	*MRR*
1	87.8%	1.25	0.92
2	88.3%	1.29	0.92
3	88.5%	1.26	0.93
4	87.3%	1.27	0.92
5	89.1%	1.24	0.93
6	88.3%	1.24	0.93
7	88.8%	1.23	0.93
8	86.5%	1.3	0.91

deteriorates from 0.93 to 0.8. Therefore, each group of features is effective and all of them are indispensable.

Furthermore, we also verify the effectiveness of each feature. As image feature set is a whole, it is unreasonable to consider it separately. Here, we only record the metrics after separately discarding each basic feature and probing feature. The results are listed in Tables 12 and 13. As shown in Tables 12 and 13, the accuracy of ARM-I will deteriorate after discarding most of features. Therefore, the most of basic and probing features affect the accuracy of ARM-I.

5.3.2 Effectiveness of the Features Used by ARM-I on Generalization. To validate the effect of features used by ARM-I on generalization, we record the metrics of ARM-I on real-world instance after discarding each group of features separately. Table 14 reports the performance of ARM-I on real-world instances. From Table 14, we can observe that all of the features used by ARM-I have effect on generalization. It is noteworthy that among three groups of features, the image feature

Table 14. Effectiveness of Three Groups of Features on Real-world Instances

Metrics	Basic features	Probing features	Image features
Precision	10.7%	71.4%	21.4%
AvgRank	3.64	1.82	3.46
MRR	0.34	0.80	0.41

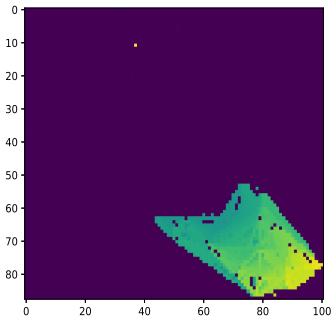


Fig. 4. The image of real-world instance t-250-d0.

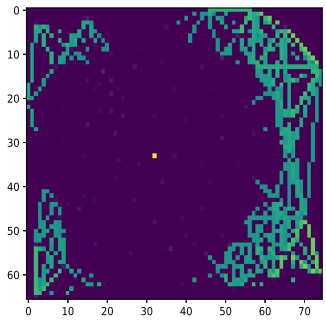


Fig. 5. The image of benchmark instance P-n101-k4.

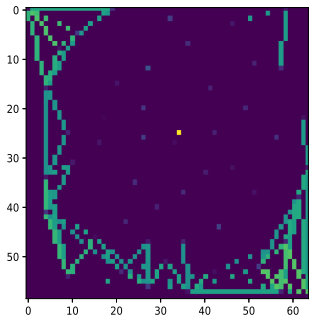


Fig. 6. The image of real-world instance E-n51-k5.

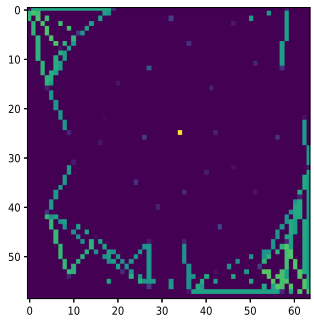


Fig. 7. The image of benchmark instance P-n45-k5.

set has the smallest effect on three metrics in benchmark instances, whereas exhibits a strong effect in real-world instances. Therefore, the proposed image features can reveal some hidden characteristics of CVRPs and increase the generalization of algorithm recommendation method.

To visually demonstrate the superiority of image features, we plot the image of real-world instance t-250-d0 in Figure 4, where the yellow point represents depot, whereas the blue points are customers. The best algorithm of t-250-d0 is IVND. In the case of just basic and probing features, the instance P-n101-k4 with 0°, 90°, and 180° rotation are its top three similar instances, of which best algorithms are all SATS. The image of P-n101-k4 is shown in Figure 5. Nevertheless, the rank of SATS for t-250-d0 is just 6th. With the addition of image features, the top three similar benchmark instances are instances E-n51-k5 and P-n45-k5 with 0° and 180° rotation, of which best algorithms are SATS, IVND, and IVND separately. The images of E-n51-k5 and P-n45-k5 are given in Figures 6 and 7. From above example, we can draw the following two observations.

Table 15. Effectiveness of Basic Features on Real-world Instances

Basic Features	*Precision*	*AvgRank*	*MRR*
1	85.7%	1.21	0.92
2	85.7%	1.21	0.92
3	85.7%	1.21	0.92
4	85.7%	1.21	0.92
5	85.7%	1.21	0.92
6	82.1%	1.25	0.9
7	78.6%	1.46	0.86
8	82.1%	1.29	0.89
9	85.7%	1.21	0.92
10	85.7%	1.21	0.92
11	17.9%	3.61	0.38
12	85.7%	1.21	0.92
13	85.7%	1.21	0.92
14	78.6%	1.75	0.84
15	85.7%	1.29	0.91
16	85.7%	1.21	0.92

Table 16. Effectiveness of Probing Features on Real-world Instances

Probing Features	*Precision*	*AvgRank*	*MRR*
1	85.7%	1.21	0.92
2	85.7%	1.21	0.92
3	85.7%	1.21	0.92
4	85.7%	1.21	0.92
5	85.7%	1.21	0.92
6	85.7%	1.21	0.92
7	85.7%	1.21	0.92
8	82.1%	1.36	0.89

First, the image features can help the method to recommend the real best algorithm for a given instance. It is clear that the inclusion of image features make the ARM-I select the best algorithm for t-250-d0.

Second, the proposed features can find some hidden characteristics of CVRPs. As plotted in Figures 5–7, the instance t-250-d0 is dramatically different from instance P-n45-k5. Nonetheless, the devised features can still capture their common characteristics, and recommend the best algorithm.

Furthermore, we also record the metrics of ARM-I after discarding each basic and probing feature, and show them in Tables 15 and 16. According to the experimental results in Tables 15 and 16, the performance of ARM-I will not be better after discarding any features, and it will deteriorate after discarding some features. In conclusion, the several basic and probing features used by ARM-I are crucial for generalization.

Table 17. The Performance of ARM-I with and without Newly Developed Features

Benchmark instances	*Precision*	*AvgRank*	*MRR*	Real-world instances	*Precision*	*AvgRank*	*MRR*
ARM-I	**89.1%**	**1.22**	**0.93**	*ARM-I*	**85.7%**	**1.21**	**0.92**
ARM-I-without	76%	1.49	0.84	*ARM-I-without*	3.6%	3.21	0.38

Table 18. The Performance of ARM-I with Different Classifiers

Benchmark instances	*Precision*	*AvgRank*	*MRR*	Real-world instances	*Precision*	*AvgRank*	*MRR*
KNN	**89.10%**	**1.22**	**0.93**	*KNN*	**85.70%**	**1.21**	**0.92**
SVM	86%	1.22	0.92	*SVM*	39.30%	2.89	0.55
MLP	49.80%	1.96	0.69	*MLP*	78.60%	1.5	0.86

It is noteworthy that although after discarding the basic feature 6, the ARM-I performs better on benchmark instances, the performance of ARM-I becomes worse on real-world instances. Taking *Precision* as an illustrate example, without basic feature 6, the performance of ARM-I on benchmark instances goes from 89.1% to 89.3%, whereas on real-world instances, it deteriorates from 85.7% to 82.1%. For an algorithm recommendation method, the performance in benchmark and real-world instances are both important. Therefore, we can draw a conclusion that each feature used in ARM-I is indispensable.

5.3.3 Effectiveness of the Proposed Features. In this article, we use three groups of features to characterize CVRP instances. For each group of features, certain new features are developed to characterize CVRP instances. To verify the effectiveness of our newly developed features, we fix the prediction algorithm and remove the newly developed features from ARM-I. The performance of ARM-I without our newly developed features is given in Table 17, which is indicated by "ARM-I-without."

In Table 17, we can find that without our newly developed features, the performance of ARM-I will considerately deteriorate, which demonstrates the effectiveness of our newly developed features. Especially, on real-world instances without our newly developed features, the *Precision* of ARM-I deteriorates to 3.6% (much less than 85.7%), the *AvgRank* deteriorates to 3.21 (much worse than 1.21), and the *MRR* deteriorates to 0.38 (much less than 0.92). It indicates that our newly developed features can significantly increase the generalization of ARM-I. Therefore, our newly developed features are effective.

5.4 The Effectiveness of KNN Classifier

In ARM-I, a KNN classifier is used to predict the best algorithm for a given CVRP instance. To verify the effectiveness of KNN classifier, we use the other two common classifiers, namely, support vector machine (SVM) and multi-layer perceptron (MLP), which have been widely used in recommendation algorithms [Chang and Lin 2011; Chu et al. 2019; Gutierrez-Rodríguez et al. 2019; Rasku et al. 2019]. The performance of ARM-I with different classifiers are shown in Table 18.

From the experimental results in Tables 7, 10, and 18, we can find that compared to existing recommendation methods, the performance ARM-I with different classifiers still rank two on benchmark instances, whereas rank one on real-world instances. It indicates that the used features can help ARM-I distinguishing different CVRP instances well, no matter what classifier it used. However, in Table 18, we can find that the ARM-I with KNN classifier achieves the best performance.

Table 19. The Performance of ARM-I with and without the Weight

Benchmark instances	*Precision*	*AvgRank*	*MRR*	Real-world instances	*Precision*	*AvgRank*	*MRR*
ARM-I	**89.1%**	**1.22**	**0.93**	*ARM-I*	**85.7%**	**1.21**	**0.92**
ARM-I-WithoutWeight	81.5%	1.35	0.88	*ARM-I-WithoutWeight*	53.6%	2.18	0.68

Table 20. The Performance of ARM-I with Different Image Feature Extraction Methods

Benchmark instances	*Precision*	*AvgRank*	*MRR*	Real-world instances	*Precision*	*AvgRank*	*MRR*
LOSIB	89.10%	1.22	0.93	LOSIB	**85.70%**	**1.21**	**0.92**
FormFactorF	79.80%	1.44	0.87	FormFactorF	10.70%	3.5	0.39
FractalDimensionF	93%	1.14	0.96	FractalDimensionF	7.10%	4	0.3
GLCMF	**90.80%**	**1.19**	**0.94**	GLCMF	7.10%	3.82	0.31
GradientF	88.70%	1.23	0.93	GradientF	46.40%	2.82	0.59
HarrisF	81.20%	1.34	0.89	HarrisF	39.30%	2.96	0.54
HuF	87.70%	1.25	0.92	HuF	14.30%	3.71	0.36
LawF	84%	1.37	0.89	LawF	10.70%	3.75	0.34
LocalBinaryPatternF	87.80%	1.25	0.92	LocalBinaryPatternF	21.40%	3.5	0.41
SURF	78.70%	1.39	0.87	SURF	14.30%	3.79	0.35

Consequently, in this article, we use KNN classifier to predict the best algorithm for a given CVRP instance.

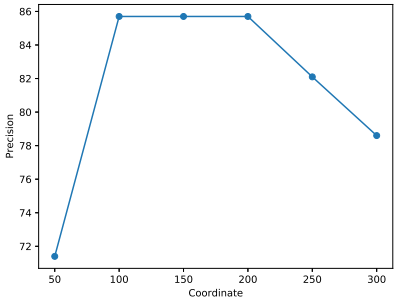
5.5 The Effectiveness of Weight

In the ARM-I, we assign the weight to basic and probing features. To demonstrate the effectiveness of the weight, we give the performance of ARM-I without the weight in Table 19, where "$ARM-I-WithoutWeight$" represents the ARM-I without the weight. In Table 19, we can find that without the weight, the performance of ARM-I will considerately deteriorate. Therefore, the weight can help the ARM-I distinguishing different types of instances, and achieve a better performance.

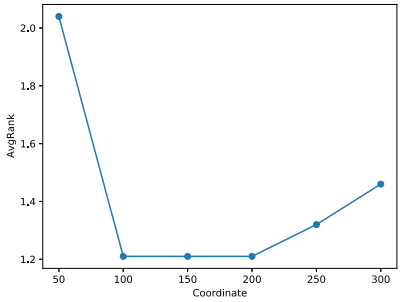
5.6 The Effectiveness of LOSIB

In ARM-I, the LOSIB is employed to extract image features. However, there are many other methods that can be used to extract image features. To test the effectiveness of LOSIB, we use the other nine image feature extraction methods including FormFactorF [Solomon and Breckon 2011], FractalDimensionF [Iftekharuddin et al. 2003], GLCMF [Haralick et al. 1973], GradientF [McGee et al. 2000], HuF [Flusser and Suk 2006], HarrisF [Harris et al. 1988], LawF [Laws 1980], LocalBinaryPatternF [Pietikäinen et al. 2011], and SURF [Bay et al. 2008]. Among these nine methods, the FormFactorF, FractalDimensionF, GLCMF, GradientF, and HuF extract global image features, whereas the HarrisF, LawF, LocalBinaryPatternF, and SURF extract local image features. Here, the codes of these extraction methods are available in Liebgott et al. [2018]. The experimental results are shown in Table 20.

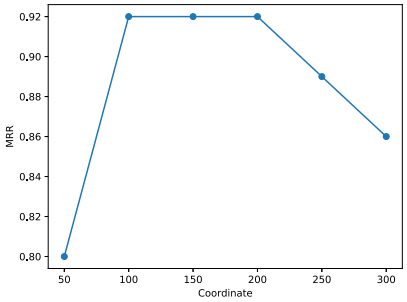
In Table 20, we can find that although using the image features extracted by FractalDimensionF and GLCMF, the ARM-I achieves better performance on benchmark instances, its performance considerately deteriorates on real-world instances. Using the image features extracted by LOSIB, the ARM-I owns a competitive performance on benchmark instances, and achieves the best performance on real-world instances. Therefore, in ARM-I, we use LOSIB to extract image features.



(a) The $Precision$ of ARM-I.



(b) The $AvgRank$ of ARM-I.



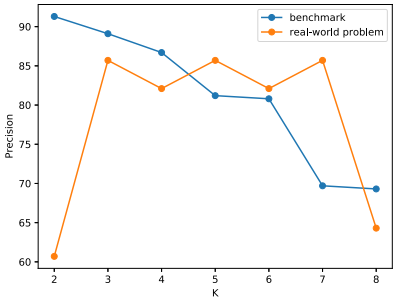
(c) The MRR of ARM-I.

Fig. 8. The performance of ARM-I under condition of converting CVRP instances into different size of rectangular images. The abscissa axis indicates the maximum values of x and y coordinates.

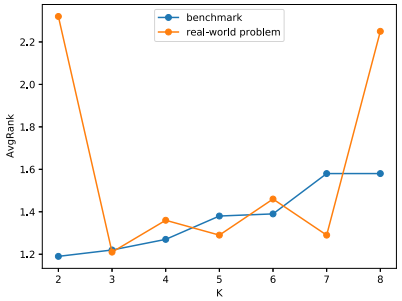
5.7 Parameters Sensitivity Analysis

5.7.1 The Size of Rectangular Image. To extract image features, the ARM-I uses the image to depict a CVRP instance. Since in real-world instance, the customer locations are represented by latitudes and longitudes, ARM-I converts the latitudes and longitudes into [0,100] before generating an image. In other words, the ARM-I uses a 100×100 image to depict real-world instance. To verify the effect of image size, we convert the real-world instances into different size of rectangular images, and the performance of ARM-I is shown in Figure 8.

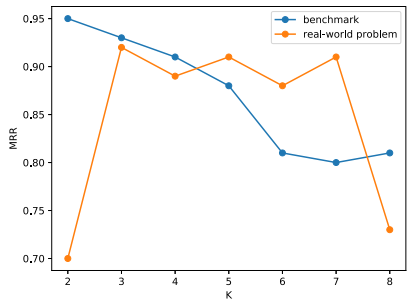
In Figure 8, we can find that the performance of ARM-I increases first and then decreases with the size of rectangular image. This is because we use the mean value of pixel differences in a certain orientation within a certain radius. When the size of rectangular image is small, the customer locations are near to each other, which leads to each image feature will consider most of customers. Therefore, the first 16 image features are similar to each other, and decrease the difference of image features between different CVRP instances. When the size of rectangular image is large, the distance between different customers is also large, which leads to each image feature will consider few customers. In this case, the first 16 image features are also similar to each other. In Figure 8, when the size of rectangular image is between 100 and 200, the ARM-I achieves the best overall performance. However, the computation of extracting the image features is increase with the size of image [García-Olalla et al. 2014]. Therefore, in this article, we use a 100×100 image to depict real-world instance.



(a) The $Precision$ of ARM-I.



(b) The $AvgRank$ of ARM-I.



(c) The MRR of ARM-I.

Fig. 9. The performance of ARM-I under different k.

5.7.2 The Parameter k. After obtaining three groups of features, the ARM-I uses a KNN classifier to predict the best algorithm for a given CVRP instance. Figure 9 depicts the performance of ARM-I under different k. From Figure 9, we can find that for the benchmark instances, the performance of ARM-I decreases with k. This is because when k is large, some instances with different best solving algorithm might be used to predict the best algorithm, which leads to the decrease of prediction accuracy. However, when k is small, only few instances are used to predict the best algorithm, which results in overfitting. Therefore, for real-world instances, the performance of ARM-I increase first and then decrease. In Figure 9, the ARM-I obtains the best overall performance when $k = 3$, Therefore, in this article, we set k to 3.

6 CONCLUSIONS AND FUTURE WORK

Several methods have been devolved to recommend suitable algorithms for different CVRP instances, where the characterization of CVRP is the first and most important step. In this article, we develop three groups of features to characterize CVRP. One group contains not only some common basic features of CVRP, but also several new designed features to depict the distribution of customer demand and the relationship between customer demand and vehicle capacity. Another group describes the sensitivity of CVRP to local search operations via the feasible, better, and worse solutions and the distribution of travel cost, instead of only feasible and better solutions. The other group uses several image features to reveal some hidden characteristics of CVRP, which is first introduced to CVRP by us. Based on the proposed features, an algorithm recommendation method called ARM-I is designed to predict the best algorithms for different CVRP instances.

In the experiments, the effectiveness of developed features has been verified, and the ARM-I has been compared to several algorithm recommendation methods designed for VRPs. According to the experimental results, the proposed three groups of features can cooperate to characterize CVRP and enhance the generalization of algorithm recommendation methods. Moreover, the ARM-I exhibits competitive performance on benchmark instances, and high generalization on real-world instances.

Despite that the features designed by us can comprehensively describe the characteristics of CVRP and the ARM-I is promising, there are still several points to be improved in the future. First, although the features are tailored for CVRP, it is believed that they have potential to be extended to more complex variants of VRPs like VRPTW. Second, the ARM-I is an off-line method, while an on-line method is also desirable to be developed. Third, in ARM-I a simple KNN classifier is used to predict the best algorithms. In the future, it is expected to design a dedicated classifier for the proposed features.

REFERENCES

Sener Akpinar. 2016. Hybrid large neighbourhood search algorithm for capacitated vehicle routing problem. *Expert Syst. Appl.* 61 (2016), 28–38.

Cigdem Alabas-Uslu and Berna Dengiz. 2011. A self-adaptive local search algorithm for the classical vehicle routing problem. *Expert Syst. Appl.* 38, 7 (2011), 8990–8998.

Jesica De Armas, Belen Melian-Batista, Jose A. Moreno-Perez, and Julio Brito. 2015. GVNS for a real-world rich vehicle routing problem with time windows. *Eng. Appl. Artif. Intell.* 42 (2015), 45–56.

Florian Arnold and Kenneth Sorensen. 2019. What makes a solution good? The generation of problem-specific knowledge for heuristics. *Comput. Operat. Res.* 106 (2019), 280–288.

Philippe Augerat. 1995. *Approche polyèdrale du problème de tournées de véhicules.* Ph.D. Dissertation. Institut National Polytechnique de Grenoble-INPG.

Herbert Bay, Andreas Ess, Tinne Tuytelaars, and Luc Van Gool. 2008. Speeded-up robust features (SURF). *Comput. Vis. Image Understand.* 110, 3 (2008), 346–359.

Mouaouia Cherif Bouzid, Hacene Aït Haddadene, and Said Salhi. 2017. An integration of Lagrangian split and VNS: The case of the capacitated vehicle routing problem. *Comput. Operat. Res.* 78 (2017), 513–525.

Juan Castro-Gutierrez, Dario Landa-Silva, and José Moreno Pérez. 2011. Nature of real-world multi-objective vehicle routing with evolutionary algorithms. In *Proceedings of the 2011 IEEE International Conference on Systems, Man, and Cybernetics.* IEEE, 257–264.

Chih-Chung Chang and Chih-Jen Lin. 2011. LIBSVM: A library for support vector machines. *ACM Trans. Intell. Syst. Technol.* 2, 3 (2011), 1–27.

Ping Chen, Hou-kuan Huang, and Xing-Ye Dong. 2010. Iterated variable neighborhood descent algorithm for the capacitated vehicle routing problem. *Expert Syst. Appl.* 37, 2 (2010), 1620–1627.

Ruey-Maw Chen, Yin-Mou Shen, and Wei-Zhi Hong. 2019. Neural-like encoding particle swarm optimization for periodic vehicle routing problems. *Expert Syst. Appl.* 138 (2019), 112833.

Xinyun Chen and Yuandong Tian. 2019. Learning to perform local rewriting for combinatorial optimization. In *Proceedings of Advances in Neural Information Processing Systems.* 6281–6292.

Nicos Christofides. 1976. The vehicle routing problem. *RAIRO-Operations Research* 10, 2 (1976), 55–70.

Nicos Christofides and Samuel Eilon. 1969. An algorithm for the vehicle-dispatching problem. *J. Operat. Res. Soc.* 20, 3 (1969), 309–318.

Xianghua Chu, Fulin Cai, Can Cui, Mengqi Hu, Li Li, and Quande Qin. 2019. Adaptive recommendation model using meta-learning for population-based algorithms. *Inf. Sci.* 476 (2019), 192–210.

Geoff Clarke and John W. Wright. 1964. Scheduling of vehicles from a central depot to a number of delivery points. *Operat. Res.* 12, 4 (1964), 568–581.

Nick Craswell. 2009. *Mean Reciprocal Rank.* Springer US, Boston, MA, 1703–1703.

George B. Dantzig and John H. Ramser. 1959. The truck dispatching problem. *Manage. Sci.* 6, 1 (1959), 80–91.

Jan Fabian Ehmke, Ann Melissa Campbell, and Barrett W. Thomas. 2016. Vehicle routing to minimize time-dependent emissions in urban areas. *Eur. J. Operat. Res.* 251, 2 (2016), 478–494.

Reza Eshtehadi, Emrah Demir, and Yuan Huang. 2020. Solving the vehicle routing problem with multi-compartment vehicles for city logistics. *Comput. Operat. Res.* 115 (2020), 104859.

Marshall L. Fisher. 1994. Optimal solution of vehicle routing problems using minimum k-trees. *Operat. Res.* 42, 4 (1994), 626–642.

Jan Flusser and Tom Suk. 2006. Rotation moment invariants for recognition of symmetric objects. *IEEE Trans. Image Process.* 15, 12 (2006), 3784–3790.

Oscar García-Olalla, Enrique Alegre, Laura Fernández-Robles, and Víctor González-Castro. 2014. Local oriented statistics information booster (LOSIB) for texture classification. In *Proceedings of the 22nd International Conference on Pattern Recognition*. IEEE, 1114–1119.

Bruce L. Golden, Subramanian Raghavan, and Edward A. Wasil. 2008. *The Vehicle Routing Problem: Latest Advances and New Challenges*. Vol. 43. Springer Science & Business Media.

Andres E. Gutierrez-Rodríguez, Santiago E. Conant-Pablos, José C. Ortiz-Bayliss, and Hugo Terashima-Marín. 2019. Selecting meta-heuristics for solving vehicle routing problems with time windows via meta-learning. *Expert Syst. Appl.* 118 (2019), 470–481.

Robert M. Haralick, Karthikeyan Shanmugam, and Its' Hak Dinstein. 1973. Textural features for image classification. *IEEE Trans. Syst. Man Cybernet.* SMC-3, 6 (1973), 610–621.

Christopher G. Harris, Mike Stephens, et al. 1988. A combined corner and edge detector. In *Proceedings of Alvey Vision Conference*, Vol. 15. Citeseer, 10–5244.

Frank Hutter, Lin Xu, Holger H. Hoos, and Kevin Leyton-Brown. 2014. Algorithm runtime prediction: Methods & evaluation. *Artif. Intell.* 206 (2014), 79–111.

Khan M. Iftekharuddin, Wei Jia, and Ronald Marsh. 2003. Fractal analysis of tumor in brain MR images. *Mach. Vis. Appl.* 13, 5–6 (2003), 352–362.

Jorge Kanda, Andre de Carvalho, Eduardo Hruschka, Carlos Soares, and Pavel Brazdil. 2016. Meta-learning to select the best meta-heuristic for the Traveling Salesman Problem: A comparison of meta-features. *Neurocomputing* 205 (2016), 393–406.

Yucheng Kao, Ming-Hsien Chen, and Yi-Ting Huang. 2012. A hybrid algorithm based on ACO and PSO for capacitated vehicle routing problems. *Mathematical Problems in Engineering* 2012, 726564 (2012), 1–17.

Pascal Kerschke, Lars Kotthoff, Jakob Bossek, Holger H. Hoos, and Heike Trautmann. 2018. Leveraging TSP solver complementarity through machine learning. *Evol. Comput.* 26, 4 (2018), 597–620.

Wouter Kool, Herke Van Hoof, and Max Welling. 2019. Attention, learn to solve routing problems! In *Proceedings of the International Conference on Learning Representations*, 1–25.

Lars Kotthoff. 2014. Algorithm selection for combinatorial search problems: A survey. *AI Mag.* 35, 3 (2014), 48–60.

Kenneth I. Laws. 1980. *Textured Image Segmentation*. Technical Report. University of Southern California Los Angeles Image Processing INST, Ph.D. Dissertation.

Jan Karel Lenstra and A. H. G. Rinnooy Kan. 1981. Complexity of vehicle routing and scheduling problems. *Networks* 11, 2 (1981), 221–227.

Feiyue Li, Bruce Golden, and Edward Wasil. 2005. Very large-scale vehicle routing: New test problems, algorithms, and results. *Comput. Operat. Res.* 32, 5 (2005), 1165–1179.

Annika Liebgott, Thomas Küstner, Heiko Strohmeier, Tobias Hepp, Philipp Mangold, Petros Martirosian, Fabian Bamberg, Konstantin Nikolaou, Bin Yang, and Sergios Gatidis. 2018. ImFEATbox: A toolbox for extraction and analysis of medical image features. *Int. J. Comput. Assist. Radiol. Surg.* 13, 12 (2018), 1881–1893.

S. Lin and B. W. Kernighan. 1973. An effective heuristic algorithm for the traveling-salesman problem. *Operat. Res.* 21, 2 (1973), 498–516.

Shih-Wei Lin, Zne-Jung Lee, Kuo-Ching Ying, and Chou-Yuan Lee. 2009. Applying hybrid meta-heuristics for capacitated vehicle routing problem. *Expert Syst. Appl.* 36, 2 (2009), 1505–1512.

Wagner Markus, Lindauer Marius, Mısır Mustafa, Nallaperuma Samadhi, and Hutter Frank. 2018. A case study of algorithm selection for the traveling thief problem. *J. Heurist.* 24, 3 (2018), 295–320.

Thomas Mayer, Tobias Uhlig, and Oliver Rose. 2018. Simulation-based autonomous algorithm selection for dynamic vehicle routing problems with the help of supervised learning methods. In *Proceedings of the 2018 Winter Simulation Conference (WSC'18)*. IEEE, 3001–3012.

Kiaran P. McGee, Armando Manduca, Joel P. Felmlee, Stephen J. Riederer, and Richard L. Ehman. 2000. Image metric-based correction (autocorrection) of motion effects: Analysis of image metrics. *J. Magn. Reson. Imag.* 11, 2 (2000), 174–181.

Olaf Mersmann, Bernd Bischl, Jakob Bossek, Heike Trautmann, Markus Wagner, and Frank Neumann. 2012. Local search and the traveling salesman problem: A feature-based characterization of problem hardness. In *Proceedings of the International Conference on Learning and Intelligent Optimization*. Springer, 115–129.

Olaf Mersmann, Bernd Bischl, Heike Trautmann, Markus Wagner, Jakob Bossek, and Frank Neumann. 2013. A novel feature-based approach to characterize algorithm performance for the traveling salesperson problem. *Ann. Math Artif. Intell.* 69, 2 (2013), 151–182.

S. A. MirHassani and N. Abolghasemi. 2011. A particle swarm optimization algorithm for open vehicle routing problem. *Expert Syst. Appl.* 38, 9 (2011), 11547–11551.

Alejandro Montoya, Christelle Guéret, Jorge E. Mendoza, and Juan G. Villegas. 2017. The electric vehicle routing problem with nonlinear charging function. *Transport. Res. B: Methodol.* 103 (2017), 87–110.

Mohammadreza Nazari, Afshin Oroojlooy, Lawrence Snyder, and Martin Takác. 2018. Reinforcement learning for solving the vehicle routing problem. In *Proceedings of Advances in Neural Information Processing Systems*. 9839–9849.

Habibeh Nazif and Lai Soon Lee. 2012. Optimised crossover genetic algorithm for capacitated vehicle routing problem. *Appl. Math. Model.* 36, 5 (2012), 2110–2117.

Mersmann Olaf, Bischl Bernd, Trautmann Heike, Wagner Markus, Bossek Jakob, and Neumann Frank. 2013. A novel feature-based approach to characterize algorithm performance for the traveling salesperson problem. *Ann. Math. Artif. Intell.* 69 (2013), 151–182.

Jorge Oyola, Halvard Arntzen, and David L. Woodruff. 2017. The stochastic vehicle routing problem, a literature review, part II: Solution methods. *EURO J. Transport. Logist.* 6, 4 (2017), 349–388.

Djamalladine Mahamat Pierre and Nordin Zakaria. 2017. Stochastic partially optimized cyclic shift crossover for multi-objective genetic algorithms for the vehicle routing problem with time-windows. *Appl. Soft Comput.* 52 (2017), 863–876.

Matti Pietikäinen, Abdenour Hadid, Guoying Zhao, and Timo Ahonen. 2011. *Computer Vision Using Local Binary Patterns*. Vol. 40. Springer Science & Business Media.

Josef Pihera and Nysret Musliu. 2014. Application of machine learning to algorithm selection for TSP. In *Proceedings of the 2014 IEEE 26th International Conference on Tools with Artificial Intelligence*. 47–54.

Jussi Rasku, Nysret Musliu, and Tommi Kärkkäinen. 2019. Feature and algorithm selection for capacitated vehicle routing problems. In *Proceedings of the European Symposium on Artificial Neural Networks, Computational Intelligence and Machine Learning*. 373–378.

Hasan Seyyedhasani and Joseph S. Dvorak. 2017. Using the vehicle routing problem to reduce field completion times with multiple machines. *Comput. Electr. Agricult.* 134 (2017), 142–150.

Kate Smith-Miles and Jano Hemert. 2011. Discovering the suitability of optimization algorithms by learning from evolved instances. *Ann. Math. Artif. Intell.* 61, 2 (2011), 87–104.

Kate A. Smith-Miles. 2009. Cross-disciplinary perspectives on meta-learning for algorithm selection. *ACM Comput. Surv.* 41, 1 (2009), 1–25.

K. A. Smith-Miles, J. I. Hemert Van, and Y. Lim. 2010. Understanding TSP dfficulty by learning from evolved instances. In *Proceedings of the 4th International Conference on Learning and Intelligent Optimization*. 266–280.

Chris Solomon and Toby Breckon. 2011. *Fundamentals of Digital Image Processing: A Practical Approach with Examples in Matlab*. John Wiley & Sons.

Xiaoqi Sun, Yuzhuo Fu, and Ting Liu. 2017. A hybrid ACO algorithm for capacitated vehicle routing problems. In *Proceedings of the 2017 IEEE 2nd Advanced Information Technology, Electronic and Automation Control Conference (IAEAC'17)*. IEEE, 510–514.

Wai Yuen Szeto, Yongzhong Wu, and Sin C. Ho. 2011. An artificial bee colony algorithm for the capacitated vehicle routing problem. *Eur. J. Operat. Res.* 215, 1 (2011), 126–135.

Boon Ean Teoh, Sivalinga Govinda Ponnambalam, and Ganesan Kanagaraj. 2015. Differential evolution algorithm with local search for capacitated vehicle routing problem. *Int. J. Bio-Inspir. Comput.* 7, 5 (2015), 321–342.

Chuan-Kang Ting, Xin-Lan Liao, Yu-Hsuan Huang, and Rung-Tzuo Liaw. 2017. Multi-vehicle selective pickup and delivery using metaheuristic algorithms. *Inf. Sci.* 406 (2017), 146–169.

Paolo Toth and Daniele Vigo. 2003. The granular tabu search and its application to the vehicle-routing problem. *Informs J. Comput.* 15, 4 (2003), 333–346.

Eduardo Uchoa, Diego Pecin, Artur Pessoa, Marcus Poggi, Thibaut Vidal, and Anand Subramanian. 2017. New benchmark instances for the capacitated vehicle routing problem. *Eur. J. Operat. Res.* 257, 3 (2017), 845–858.

Jano I. van Hemert. 2005. Property analysis of symmetric travelling salesman problem instances acquired through evolution. In *Proceedings of the European Conference on Evolutionary Computation in Combinatorial Optimization*. Springer, 122–131.

F. Yu Vincent, A. A. N. Perwira Redi, Chao-Lung Yang, Eki Ruskartina, and Budi Santosa. 2017. Symbiotic organisms search and two solution representations for solving the capacitated vehicle routing problem. *Appl. Soft Comput.* 52 (2017), 657–672.

David H. Wolpert and William G. Macready. 1997. No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* 1, 1 (1997), 67–82.

Wai Keung Wong, Zhihui Lai, Jiajun Wen, Xiaozhao Fang, and Yuwu Lu. 2017. Low-rank embedding for robust image feature extraction. *IEEE Trans. Image Process.* 26, 6 (2017), 2905–2917.

Yu Chieh Wu and Jie Chi Yang. 2008. A robust passage retrieval algorithm for video question answering. *IEEE Trans. Circ. Syst. Vid. Technol.* 18, 10 (2008), 1411–1421.

Jianhua Xiao, Tao Zhang, Jingguo Du, and Xingyi Zhang. 2019. An evolutionary multiobjective route grouping-based heuristic algorithm for large-scale capacitated vehicle routing problems. *IEEE Trans. Cybernet.* In Press. DOI: 10.1109/TCYB.2019.2950626

Weixiong Zhang and Richard E. Korf. 1996. A study of complexity transitions on the asymmetric traveling salesman problem. *Artif. Intell.* 81, 1–2 (1996), 223–239.

Zhao Zhang, Fanzhang Li, Mingbo Zhao, Li Zhang, and Shuicheng Yan. 2017. Robust neighborhood preserving projection by nuclear/l2, 1-norm regularization for image feature extraction. *IEEE Trans. Image Process.* 26, 4 (2017), 1607–1622.

Wenzhi Zhao and Shihong Du. 2016. Spectral–spatial feature extraction for hyperspectral image classification: A dimension reduction and deep learning approach. *IEEE Trans. Geosci. Remote Sens.* 54, 8 (2016), 4544–4554.

Yongquan Zhou, Qifang Luo, Jian Xie, and Hongqing Zheng. 2016. A hybrid bat algorithm with path relinking for the capacitated vehicle routing problem. In *Proceedings of the Metaheuristics and Optimization in Civil Engineering*. Springer, 255–276.

Çağrı Koç, Tolga Bektaş, Ola Jabali, and Gilbert Laporte. 2016. Thirty years of heterogeneous vehicle routing. *Eur. J. Operat. Res.* 249, 1 (2016), 1–21.

Received June 2020; revised November 2020; accepted January 2021